

DS 4420 - Machine Learning 2

Research Question: Predicting Steam Game Prices Using Machine Learning

Submitted by:
Gianna Saw
Jason Zheng
Section 01

1. Introduction and Literature Review

1.1. Problem Statement

The project is centered on the following question: **to what extent can a game's content, quality, and engagement metrics be used to predict its price on the Steam marketplace?**

This question lies at the intersection of digital platform economics and machine learning. Pricing decisions on large global platforms such as Steam have significant implications for independent developers, as price influences both market positioning and consumer demand.

Steam is a digital game distribution platform developed by Valve Corporation. The site hosts over 50,000 games from both major publishers and independent developers. As the dominant PC gaming marketplace, Steam provides rich metadata for each game title including pricing, review scores, genre tags, and playtime statistics, which makes it a valuable source for studying digital goods pricing behavior (Knight, 2022).

Pricing is a critical decision for game developers, especially for independent studios with limited marketing budgets. On Steam, price signals quality and positions a game relative to competitors. For case in point: too high and the game risks low adoption; too low and it may undermine perceived value. Unlike physical goods, digital games have near-zero marginal cost (Marchand & Hennig-Thurau, 2013). This means that pricing is driven almost entirely by perceived value rather than production cost. Understanding which game characteristics are associated with higher prices can therefore help developers make more informed launch decisions.

1.2. Literature Context

Recent research has begun applying machine learning techniques to Steam data to understand platform market dynamics better. For instance, Du's 2021 study uses Steam data to predict whether a game will receive a discount using several classification methods. Ultimately, the study found that Random Forest outperforms Logistic Regression (79.5% accuracy) for prediction outcomes. This work initially supports the idea that machine learning models can successfully capture patterns in Steam game metadata related to pricing strategies. However, the study frames pricing as a binary classification problem rather than attempting to model the price itself as a continuous outcome. This is an idea that can be expanded.

Similarly, Luisa et al (2021) explores Steam data. However, this study now uses a Bayesian hierarchical model to predict game popularity on the platform. Their analysis found that variables such as genre, price, supported languages, and release timing significantly influence player counts. The study treats price as the explanatory variable for popularity and concludes that pricing decisions interact closely with observable game characteristics. In contrast to their framework, our study reverses this relationship by treating price as the outcome variable and examining which metadata features can be used to predict it.

While studies using Steam data have largely focused on predicting popularity or promotional behavior, research in other industries has explored the direct prediction of product prices. Stubnarova et al. (2020) compare neural networks and hedonic regression models for predicting residential property prices. Their findings show that neural networks trained with Bayesian

regularization outperform traditional regression approaches, significantly improving prediction accuracy. These results suggest that nonlinear machine learning models can capture complex relationships between product attributes and price more effectively than classical regression techniques. This similar methodology motivates our approach: we apply a similar model comparison framework to a digital goods marketplace, where product characteristics are well documented but pricing dynamics remain less understood.

Similar approaches have also been applied within the video game industry itself. Li et al (2021)'s work investigated video games sales using neural networks. Features such as genre, publisher, and release timing were in conjunction with ReLU activation functions to model relationships between game metadata and economic outcomes. The success of neural architectures in predicting game performance supports the use of a multilayer perceptron (MLP) in our study for price regression. However, we extend this approach by implementing the neural network manually from scratch. This ultimately allows us to examine the modeling process and training dynamics more closely.

Beyond prediction tasks, a broader body of research has sought to understand the drivers of success in the digital game marketplace, as seen in Ma et al (2025)'s research. The study analyzed game performance across multiple platforms, including Steam, Twitch, and Metacritic. The study found that Steam prelaunch engagement metrics were the strongest predictors of game revenue. On the other hand, Metacritic scores had a smaller but statistically significant effect. While Ma et al. focus on revenue rather than listed price, their feature importance findings inform our feature selection, specifically around engagement metrics and review scores.

Taken together, these studies highlight two important trends in existing literature. First, machine learning methods have increasingly been applied to Steam data to analyze platform behavior, particularly in predicting game popularity or promotional strategies. Second, price prediction has been studied extensively in other areas. These common areas include neural networks and regression models used to model relationships between product attributes and market value. However, relatively little research has directly examined whether the price of digital games itself can be predicted from the metadata available on distribution platforms such as Steam.

1.3. Our Approach

Our study addresses this gap by directly modeling game price as a continuous outcome using both a manual MLP and a Bayesian linear regression model. This allows us to compare predictive performance with interpretability.

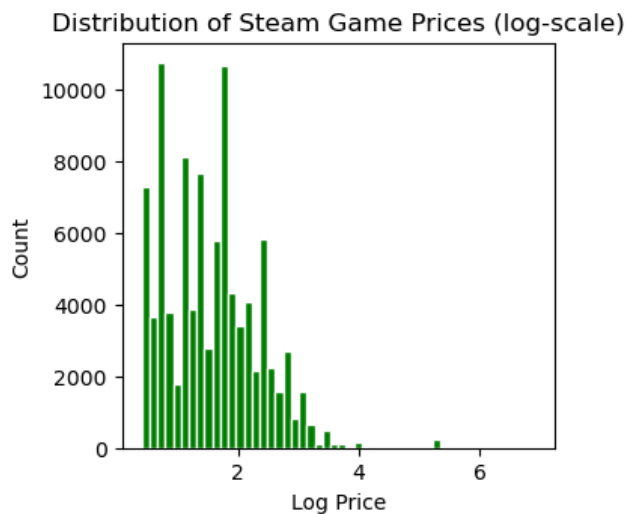
2. ML Methodology and Dataset

2.1. Dataset

The project uses the [FronkonGames Steam Games Dataset](#), which can be accessed via the Hugging Face datasets library. Data has been scraped using Steam API. It has 120k data rows, with 41 features including pricing, review statistics, platform support, genre classifications, and engagement metrics such as playtime and peak concurrent users. After preprocessing and filtering to paid games only, the final dataset contained 97,734 observations.

The figure below shows the distribution of Steam game prices, in a log-scale format.

Figure 1: Distribution of Steam game prices, log-scaled



2.2. Preprocessing

Several preprocessing steps were also applied to prepare the data for modeling:

- Non-predictive columns (i.e. game descriptions, developer names, URLs, and media links) were removed,
- The supported_languages column was stored as a numpy array and was converted to a numeric language_count feature by computing the length of each array,
- The estimated_owners column contained range strings such as "0–20000" and was parsed by taking the midpoint of each range,
- Genre columns were one-hot encoded from list-type fields using pd package,
- Highly skewed engagement features (positive, negative, recommendations, peak_ccu, estimated_owners, and all playtime columns) were log-transformed using log1p to reduce skewness,
- The target variable price was similarly log-transformed and free games (price = 0 after transformation) were removed; only paid titles were left in the dataset.

For Model 1, three additional engineered features were constructed:

- review_ratio (positive reviews as a proportion of total reviews),
- engagement (average of mean and median playtime forever),
- and platform_count (sum of Windows, Mac, and Linux support flags)

All features were min-max scaled using training data only before being passed to the model.

2.3. Model 1: Manual MLP via Python

To predict game price from metadata, we built a multilayer perceptron from scratch using NumPy. The network has one hidden layer with 64 units, a ReLU activation, and a linear output layer. As for weights, these were initialised from $N(0, 0.1)$. The model was trained using full-batch gradient descent with a learning rate of 0.001 and MSE loss. Additionally, these gradients were clipped at ± 5.0 to prevent instability during training. We ran 700 epochs on an 80/20 train/test split.

2.3.1. Segmented MLP

The residuals from the initial single MLP showed that prediction errors were much larger for higher-priced games. This indicates that one model was not enough to capture the full pricing distribution. Cheap indie games and expensive AAA titles are priced very differently, so training a single model forces it to find a compromise that works poorly for both ends.

To fix this, we split the training data at the median log price and trained 2 separate MLPs (one lower-priced; one high-priced games). As for testing, each game title gets routed to the right model based on the same threshold and predictions are combined. In doing so, the model can focus on its own segment rather than trying to generalize across the full price range.

2.4. Model 2: Bayesian Multinomial Logistic Regression via R

For the second model, we shifted from predicting price as now a classification problem. We binned games into four price tiers based on their log-transformed price:

- Low (below $\log(6)$),
- Mid ($\log(6)$ to $\log(16)$),
- High ($\log(16)$ to $\log(31)$),
- and Premium (above $\log(31)$).

This makes sense, given the clustered price distribution we saw in the EDA. Steam games tend to group around familiar price points like \$4.99, \$9.99, and \$29.99 rather than being spread continuously.

We fit a Bayesian multinomial logistic regression using the `MCMCmnl` function from the `MCMCpack` library in R. The predictors were `peak_ccu`, `recommendations`, `positive`, `negative`, `metacritic_score`, `achievements`, `average_playtime_forever`, and `language_count`. The sampler ran for 5,000 iterations with a 1,000 iteration burn-in and thinning factor of 5.

The main advantage of using Bayesian is that it results in full posterior distributions over each coefficient instead of just point estimates. This allows for meaningful insights regarding the uncertainty. Specifically, which features have credible effect on price tier and which do not. To generate predictions on the test set and build a confusion matrix, we also fit a frequentist multinomial model using the nnet package on the same features.

3. Results and Interpretation

3.1. Evaluation Metrics

Model 1 was evaluated using MSE, RMSE, and R^2 on the held-out test set, reported separately for the single MLP, the low-price segment, the high-price segment, and the combined segmented model. Model 2 was evaluated using overall classification accuracy and class-wise accuracy from the confusion matrix.

3.2. Model 1: Manual MLP

Model	MSE	RMSE	R^2
Single MLP	0.675	0.8218	-0.0849
Low-Price MLP	0.1337	0.3657	-0.1752
High-Price MLP	0.4081	0.6389	-0.3175
Segmented MLP	0.2714	0.5209	0.5640

Figure 2: Predicted vs actual and residuals for single MLP (left) and segmented MLP (right)

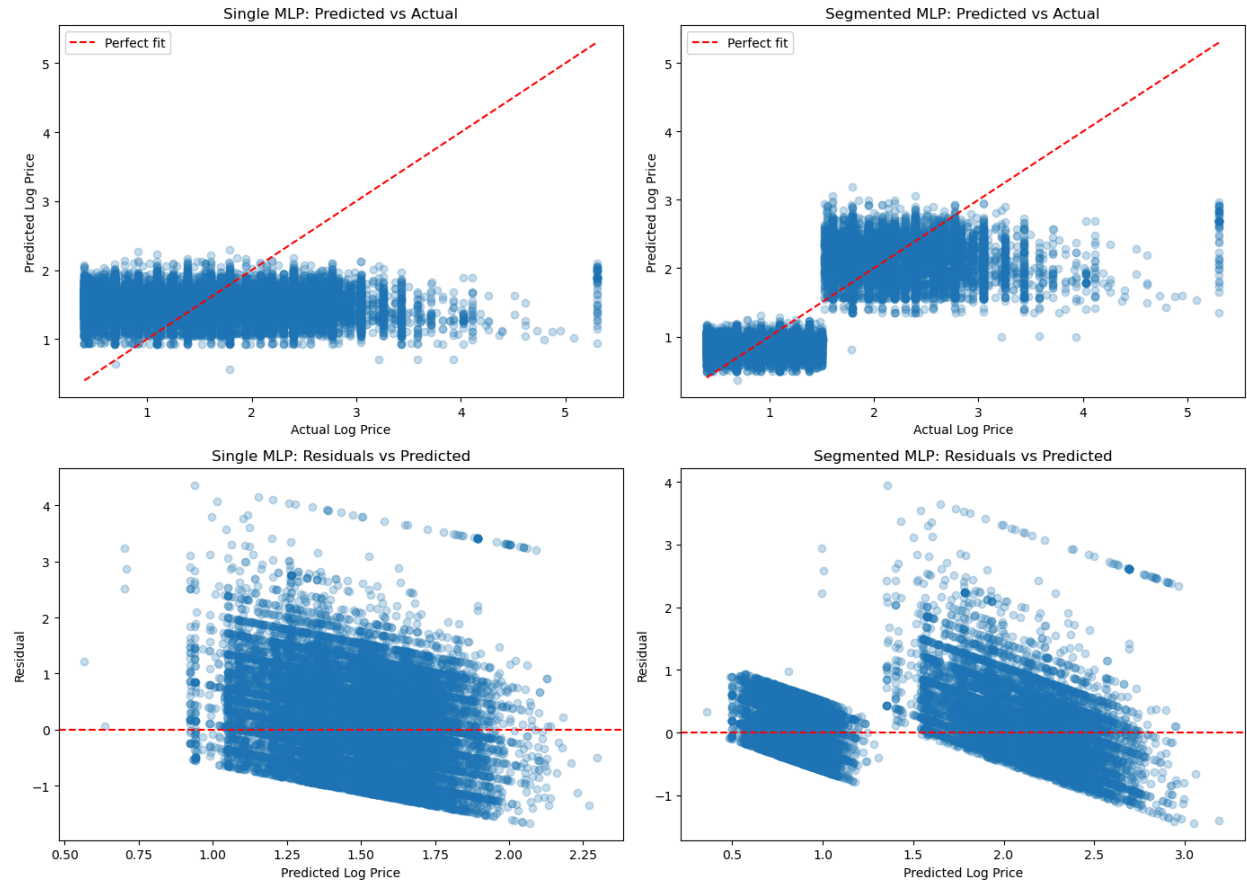
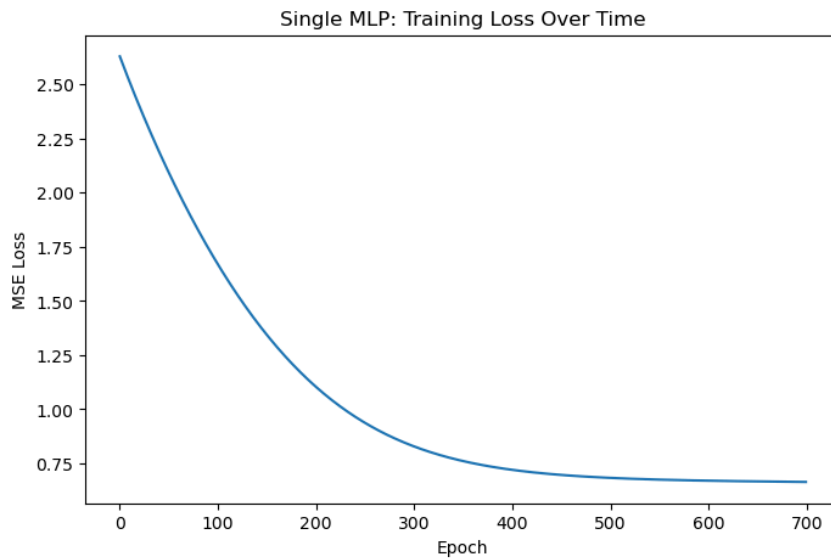
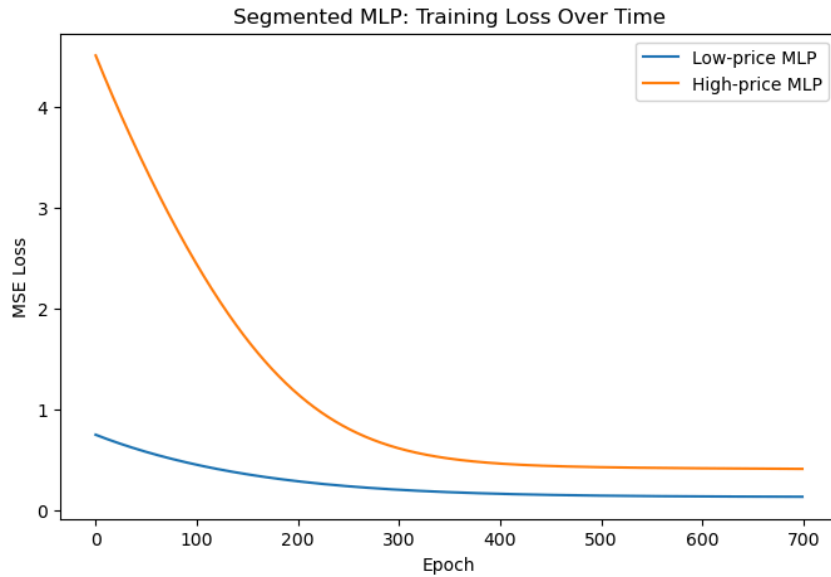


Figure 3: Training loss over 700 epochs for the low-price and high-price segment and single MLPs



The single MLP achieved an R^2 of -0.085 which is worse than just predicting the mean price for every game. Looking at Figure 2 (top left), predictions collapse into a narrow band around log price 1.5–2.0 regardless of actual value. Training loss did drop steadily from 2.63 to 0.67 over 700 epochs, so the model was learning something, but one network could not handle the full price range.

Splitting at the median log price of 1.52 and training two separate MLPs brought the combined R^2 up to 0.564. Figure 2 (top right) shows two distinct clusters that both track the diagonal much more closely than before. The low-price MLP converged from MSE 0.748 to 0.135, and the high-price MLP from 4.51 to 0.412 (both shown in Figure 3).

The individual segment R^2 values are both negative (-0.175 and -0.318), which looks odd at first. This is because each segment covers a narrow price range, so R^2 is computed against a very tight mean (even decent predictions score poorly independently). The combined R^2 of 0.564 comes from the segmented model correctly separating the two regimes, which the single model never did.

The residual plots (Figure 1, bottom row) reinforce this. The single MLP shows a strong downward slope. For instance, we see large positive residuals at low predicted prices and near-zero at high predicted prices, which is a classic sign of systematic underprediction. The segmented residuals are much more centered around zero, though some fan shape remains in the high-price segment. This indicates that expensive games are still the harder ones to pin down.

3.3. Model 2: Bayesian Multinomial Logistic Regression Results

Figure 4: Confusion matrix showing class-wise prediction proportions

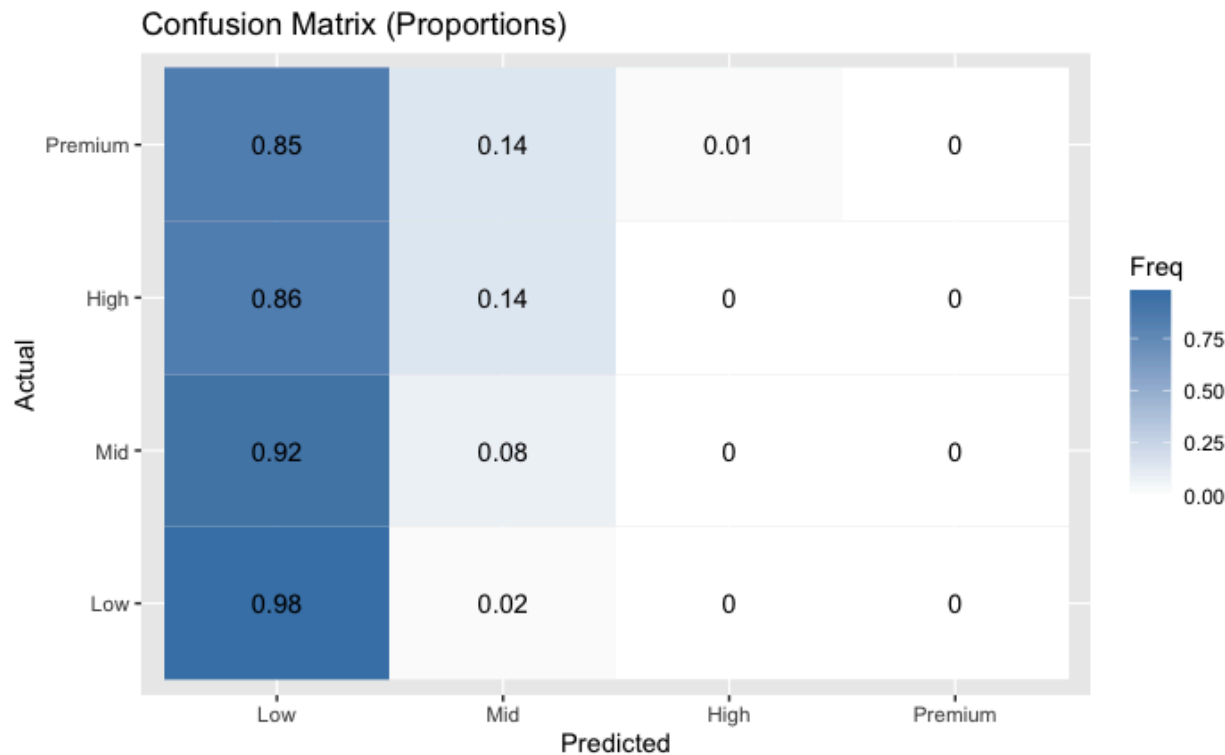
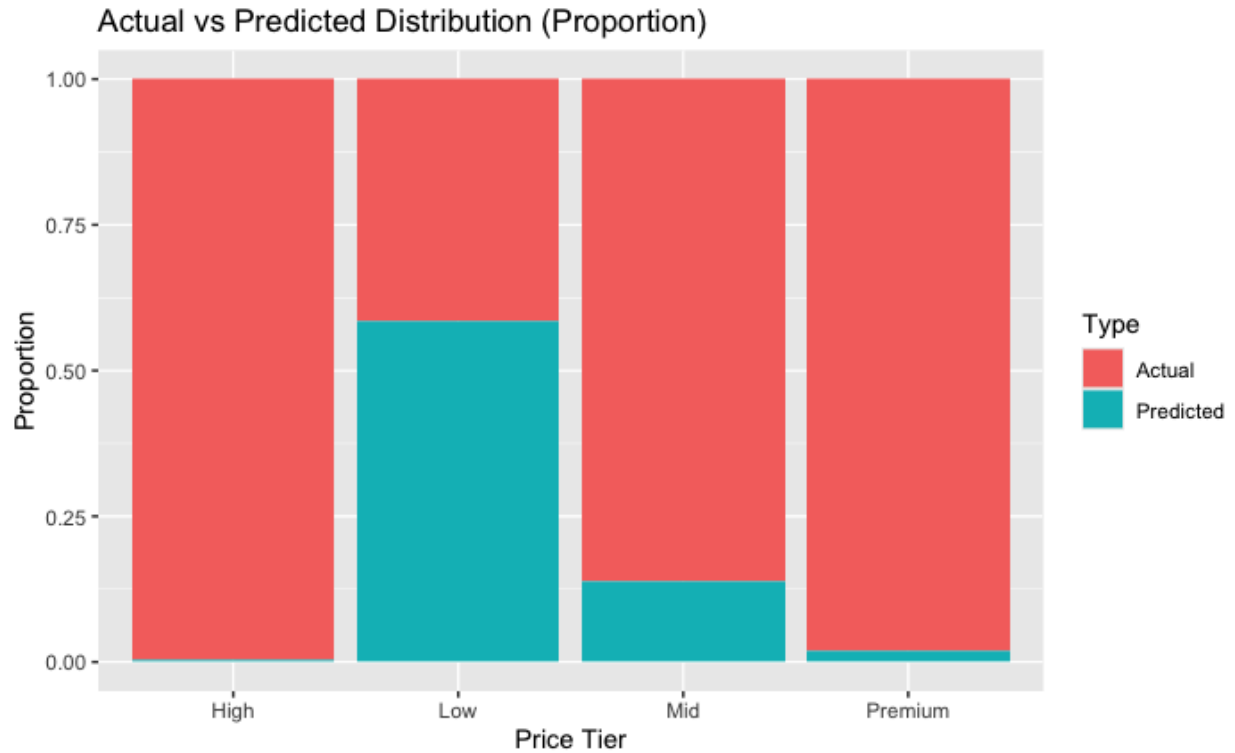


Figure 5: Actual vs predicted distribution by price tier



The model achieved an overall classification accuracy of 68.3% on the test set. Given that 67.6% of games fall in the Low tier alone, this is only marginally better than a naive majority-class classifier. Ultimately, this tells us the model is largely predicting Low for most games.

The confusion matrix confirms this. The model correctly classified 98% of Low-tier games and 92% of Mid-tier games, but almost entirely failed on High (0.2% correct) and Premium (0% correct) games. This reflects the severe class imbalance in the dataset. For instance, Low tier makes up 67.6% of games, Mid 26.8%, High 4.4%, and Premium just 1.1%. With so few examples of expensive games, the model never learned what distinguishes them.

Figure 6: Bayesian Coefficient Estimates Across Price Tier

```

##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##              Mean          SD Naive SE Time-series SE
## (Intercept).Low      2.6719678 0.0445230 1.408e-03    1.761e-02
## (Intercept).Mid      1.8515210 0.0348887 1.103e-03    9.683e-03
## (Intercept).Premium  -1.2159733 0.0429992 1.360e-03    8.305e-03
## peak_ccu.Low         -0.4887065 0.0150741 4.767e-04    3.037e-03
## peak_ccu.Mid         -0.1611714 0.0129558 4.097e-04    2.126e-03
## peak_ccu.Premium      0.2796043 0.0289591 9.158e-04    9.543e-03
## recommendations.Low  -0.1738083 0.0108137 3.420e-04    2.726e-03
## recommendations.Mid  -0.0763650 0.0091137 2.882e-04    1.864e-03
## recommendations.Premium -0.1007254 0.0397840 1.258e-03    2.526e-02
## positive.Low         0.3177706 0.0254593 8.051e-04    8.027e-03
## positive.Mid         0.2480701 0.0196105 6.201e-04    4.323e-03
## positive.Premium     -0.3808677 0.0800854 2.533e-03    2.611e-02
## negative.Low         -0.1171456 0.0240396 7.602e-04    7.091e-03
## negative.Mid         -0.2119183 0.0219839 6.952e-04    4.595e-03
## negative.Premium      0.3510670 0.0855031 2.704e-03    2.523e-02
## metacritic_score.Low  0.0063860 0.0005792 1.831e-05    9.682e-05
## metacritic_score.Mid  0.0029956 0.0005734 1.813e-05    7.314e-05
## metacritic_score.Premium 0.0017262 0.0027135 8.581e-05    9.278e-04
## achievements.Low      0.0005373 0.0003413 1.079e-05    1.309e-04
## achievements.Mid      0.0001168 0.0003042 9.621e-06    9.486e-05
## achievements.Premium  -0.0158060 0.0019514 6.171e-05    4.522e-04

##
## average_playtime_forever.Low -0.0884469 0.0105566 3.338e-04    2.224e-03
## average_playtime_forever.Mid -0.0406498 0.0086442 2.734e-04    1.466e-03
## average_playtime_forever.Premium 0.0001191 0.0262279 8.294e-04    6.860e-03
## language_count.Low      0.0198323 0.0017006 5.378e-05    3.268e-04
## language_count.Mid      0.0058411 0.0014663 4.637e-05    1.659e-04
## language_count.Premium  0.0404367 0.0016558 5.236e-05    2.776e-04
##

```

The Bayesian MCMC output gives us something the confusion matrix cannot, which is the posterior distributions over each coefficient. A few interesting findings from the posterior means: positive reviews had a positive effect on Low and Mid tiers ($\beta \approx 0.32$ and 0.25) but a negative effect on Premium ($\beta \approx -0.38$). This does suggest that highly reviewed games tend to be cheaper. Peak CCU had a negative effect on Low and Mid but positive on Premium, meaning games with high concurrent players at peak are more likely to be expensive. Language count had a small but consistently positive effect across all tiers, which suggests games available in more languages tend to be priced higher.

4. Conclusions and Future Work

4.1. Summary of Findings

This study examined whether Steam game prices can be predicted from publicly available metadata. We found that a single MLP struggled to generalize across the full price distribution, achieving an R^2 of -0.085. Segmenting the data at the median price and training two separate MLPs significantly improved performance, which brought the combined R^2 to 0.564. This suggests that cheap and expensive games follow different pricing dynamics that a unified model cannot simultaneously capture.

On the other hand, the Bayesian multinomial logistic regression achieved 68.3% overall accuracy. However, this was largely driven by correctly classifying Low-tier games, which dominate the dataset. Where the Bayesian model added value was in its posterior distributions, we found that positive reviews and language count had credible positive effects on price tier, while games with high peak CCU were more likely to fall in the Premium tier. These findings align with the intuition that popular, widely localized games tend to command higher prices. This makes sense as Steam ecosystem of games is extremely competitive and classifying what makes higher tier game is and should be more complex.

4.2. Limitations

Both models carry assumptions worth discussing. The MLP assumes that the selected features capture the relevant pricing signal, but a significant driver of high-end pricing like publisher reputation and brand is absent from the dataset. The Bayesian model assumes that the relationship between predictors and price tier is linear on the log-odds scale, which may not hold across all tiers. The severe class imbalance (67.6% Low, 1.1% Premium) also meant the Bayesian model never saw enough examples of expensive games to learn their distinguishing characteristics.

The `estimated_owners` feature was also heavily bucketed. For instance, most games shared the same midpoint value, which limited its predictive power. Additionally, the manual MLP used full-batch gradient descent, which is slower and less stable than mini-batch optimization. This likely contributed to the individual segment R^2 values being negative even when combined performance was strong.

4.3. Future Work

A few directions could meaningfully improve on this work. First, framing the problem as price tier classification from the start, rather than regression, may yield better results for both MLP and Bayesian. This is something we found in the latter half of the project; price distribution naturally clusters around common price points. Second, adding publisher-level features or embeddings could help the model learn the brand effects that currently drive high-end pricing. Third, replacing full-batch gradient descent with mini-batch optimization or using a library like PyTorch would make training more stable and allow for deeper networks.

5. Appendix

- [Link to Github](#)
- [Link to working website](#)
- The python function below can be used to fit our model:

```
# DS 4420 - Model 1: Segmented MLP (Manual)
# Authors: Gianna Saw, Jason Zheng

import pandas as pd
import numpy as np

# -----
# 1) Load preprocessed dataset
# -----
steam = pd.read_csv("steam_clean_v2.csv")

# Keep only top 5 genre dummies
genre_cols = [col for col in steam.columns if col.startswith("genre_")]
top_genres = steam[genre_cols].sum().sort_values(ascending=False).head(5).index.tolist()
drop_genres = [col for col in genre_cols if col not in top_genres]
steam = steam.drop(columns=drop_genres)

# -----
# 2) Feature engineering
# -----
steam["review_ratio"] = np.where(
    (steam["positive"] + steam["negative"]) > 0,
    steam["positive"] / (steam["positive"] + steam["negative"]),
    0.5
)

steam["engagement"] = (
    steam["average_playtime_forever"] + steam["median_playtime_forever"]
) / 2

steam["platform_count"] = steam["windows"] + steam["mac"] + steam["linux"]

# -----
# 3) Train split + scaling
# -----
steam = steam.sample(frac=1, random_state=42).reset_index(drop=True)
```

```

split = int(0.8 * len(steam))
train = steam.iloc[:split]

X_train = train.drop(columns=["price"]).values
y_train = train["price"].values.reshape(-1, 1)

# Min-max scale using training only
X_min = X_train.min(axis=0)
X_max = X_train.max(axis=0)
X_train_scaled = (X_train - X_min) / (X_max - X_min + 1e-8)

# Add bias feature
X_train_scaled = np.hstack([X_train_scaled, np.ones((X_train_scaled.shape[0], 1))])

# -----
# 4) Manual MLP implementation
# -----
def relu(z):
    return np.maximum(0, z)

def train_mlp_manual(X_train_seg, y_train_seg, H=64, eta=0.001, epochs=700, seed=1,
clip_value=5.0):
    np.random.seed(seed)

    p = X_train_seg.shape[1]
    W1 = np.random.normal(0, 0.1, (p, H))
    W2 = np.random.normal(0, 0.1, (H, 1))

    errors = []

    for epoch in range(epochs):
        grad_W2 = np.zeros_like(W2)
        grad_W1 = np.zeros_like(W1)
        loss = 0.0

        for i in range(X_train_seg.shape[0]):
            x = X_train_seg[i].reshape(-1, 1)
            yi = y_train_seg[i].reshape(1, 1)

            # forward pass
            z1 = W1.T @ x
            h = relu(z1)
            f = W2.T @ h

```

```

# squared error
error = f - yi
loss += error.item() ** 2

# manual backprop
grad_W2 += h @ error
indicator = (z1 > 0).astype(int)
back = (W2 @ error) * indicator
grad_W1 += x @ back.T

# average gradients
grad_W2 /= X_train_seg.shape[0]
grad_W1 /= X_train_seg.shape[0]

# clip gradients
grad_W2 = np.clip(grad_W2, -clip_value, clip_value)
grad_W1 = np.clip(grad_W1, -clip_value, clip_value)

# gradient descent update
W2 -= eta * grad_W2
W1 -= eta * grad_W1

mse = loss / X_train_seg.shape[0]
errors.append(mse)

return W1, W2, errors

# -----
# 5) Train single MLP
# -----
W1_single, W2_single, errors_single = train_mlp_manual(
    X_train_seg=X_train_scaled,
    y_train_seg=y_train,
    H=64,
    eta=0.001,
    epochs=700,
    seed=1
)

# -----
# 6) Train segmented MLPs
# -----

```

```

threshold = np.median(y_train)

low_mask_train = y_train.flatten() < threshold
high_mask_train = y_train.flatten() >= threshold

X_train_low = X_train_scaled[low_mask_train]
y_train_low = y_train[low_mask_train]

X_train_high = X_train_scaled[high_mask_train]
y_train_high = y_train[high_mask_train]

W1_low, W2_low, errors_low = train_mlp_manual(
    X_train_seg=X_train_low,
    y_train_seg=y_train_low,
    H=64,
    eta=0.001,
    epochs=700,
    seed=1
)

W1_high, W2_high, errors_high = train_mlp_manual(
    X_train_seg=X_train_high,
    y_train_seg=y_train_high,
    H=64,
    eta=0.001,
    epochs=700,
    seed=1
)

```

- The R function below can be used to fit our model:

```

---
title: "DS4420 - Steam Game Price Prediction: Bayesian Linear Regression"
author: "Gianna Saw and Jason Zheng"
date: "2026-04-16"
output:
  pdf_document: default
  html_document: default
---

```${r setup}
library(MCMCpack)

```



```

library(nnet)

steam_df <- read.csv("steam_clean_v2.csv")

Standardize column names
names(steam_df) <- tolower(names(steam_df))
names(steam_df) <- gsub("\\.", "_", names(steam_df))

Create price tiers (log-price scale from preprocessing pipeline)
steam_df$price_tier <- cut(
 steam_df$price,
 breaks = c(-Inf, log(0 + 1), log(5 + 1), log(15 + 1), log(30 + 1), Inf),
 labels = c("Free", "Low", "Mid", "High", "Premium"),
 right = TRUE
)
steam_df$price_tier <- as.factor(steam_df$price_tier)

Keep rows complete for model variables
keep_vars <- c(
 "price_tier", "peak_ccu", "recommendations", "positive", "negative",
 "metacritic_score", "achievements", "average_playtime_forever", "language_count"
)
keep_vars <- keep_vars[keep_vars %in% names(steam_df)]
steam_df <- steam_df[complete.cases(steam_df[, keep_vars]),]

Train/test split (used for model training workflow consistency)
set.seed(123)
n <- nrow(steam_df)
train_index <- sample(1:n, size = round(0.8 * n))
train_df <- steam_df[train_index,]
test_df <- steam_df[-train_index,]

Bayesian multinomial logistic regression (MCMCpack)
bayes_model <- MCMCmnl(
 price_tier ~ peak_ccu + recommendations + positive + negative +
 metacritic_score + achievements + average_playtime_forever + language_count,
 data = train_df,
 mcmc = 5000,
 burnin = 1000,
 thin = 5,
 verbose = 0
)

```

```
summary(bayes_model)

Frequentist multinomial model (training implementation comparator)
train_df$price_tier <- droplevels(train_df$price_tier)
multinom_model <- multinom(
 price_tier ~ peak_ccu + recommendations + positive + negative +
 metacritic_score + achievements + average_playtime_forever + language_count,
 data = train_df,
 trace = FALSE
)
```

## 6. **References**

- De Luisa, Andraž, Jan Hartman, David Nabergoj, Samo Pahor, Marko Rus, Bozhidar Stevanoski, Jure Demšar, and Erikštrumbelj. 2021. "Predicting the Popularity of Games on Steam." *ELEKTROTEHNIŠKI VESTNIK* 88 (4): 151–62.  
<https://ev.fe.uni-lj.si/4-2021/Luisa.pdf>.
- Du, Lingyu. 2021. "Steam Game Discount Prediction Using Machine Learning Methods." *2021 3rd International Conference on Machine Learning, Big Data and Business Intelligence (MLBDI)*, December, 149–52.  
<https://doi.org/10.1109/mlbdi54094.2021.00037>
- Knight, R. (2022, March 9). What is Steam? - LoveThyNerd.com. Love Thy Nerd.  
<https://lovethynerd.com/what-is-steam/>
- Li, Jianbin, Yufan Zheng, Haoran Hu, Junhui Lu, and Choujun Zhan. 2021. "Predicting Video Game Sales Based on Machine Learning and Hybrid Feature Selection Method." *2021 16th International Conference on Intelligent Systems and Knowledge Engineering (ISKE)*, November.  
<https://doi.org/10.1109/iske54062.2021.9755343>.
- Ma, Jiesi. 2025. "Unveiling Success Drivers in Gaming: A Machine Learning Study across Steam, Twitch, and Metacritic." Edited by Michael J. Katchabaw. *International Journal of Computer Games Technology* 2025 (1).  
<https://doi.org/10.1155/ijcg/5776632>.
- Marchand, A., & Hennig-Thurau, T. (2013). Value Creation in the Video Game Industry: Industry Economics, Consumer Benefits, and Research Opportunities. *Journal of Interactive Marketing*, 27(3), 141–157. <https://doi.org/10.1016/j.intmar.2013.05.001>
- Stubnarova Michaela, Marta Urbaníková, Jarmila Hudáková, and Viera Papcunová. 2020. "Estimation of Residential Property Market Price: Comparison of Artificial Neural

Networks and Hedonic Pricing Model.” *Emerging Science Journal* 4 (6): 530–38.  
<https://doi.org/10.28991/esj-2020-01250>.